

1주차

📅 날짜	2026년 9월 8일
≡ 텍스트	파머완 1장

1. 파이썬 기반의 머신러닝과 생태계 이해

01. 머신러닝의 개념

머신러닝 - 애플리케이션 수정 없이 데이터 기반으로 패턴 학습&결과 예측하는 알고리즘 기법

- 기존 소프트웨어 코드만으로 해결하기 어려웠던 문제들 해결가능
 - 현실세계의 매우 복잡한 조건 or 업무적으로 복잡한 조건/규칙들이 다양한 형태로 결합&시시각각 변함 → 일정한 패턴 찾기 어려운 경우
- 데이터 내 숨겨진 패턴 스스로 인지 + 신뢰도 있는 예측결과 도출
 - 데이터 기반 → 통계적 신뢰도 강화
 - 다양한 수학적 기법 → 예측 오류 최소화

1) 머신러닝의 분류

- 지도학습(Supervised Learning)
 - 분류
 - 회귀
 - 추천 시스템
 - 시각/음성감지/인지
 - 텍스트 분석, NLP
- 비지도학습(Un-supervised Learning)
 - 클러스터링
 - 차원 축소
 - 강화학습

2) 데이터 전쟁

- 머신러닝 알고리즘도 중요하지만 **데이터**의 중요성이 무엇보다 커짐
- 머신러닝 가장 큰 단점 - 데이터 매우 의존적
가비지 인(Garbage In), 가비지 아웃(Garbage Out)
: 좋은 품질 데이터 갖추지 ❌ → 머신러닝 수행 결과 📉

데이터 이해하고 효율적으로 가공, 처리, 추출 → 최적의 데이터 준비하는 능력 중요
어떠한 품질의 데이터로 만든 머신러닝 모델이냐가 앞으로 많은 회사의 경쟁력 결정할수도

3) 파이썬과 R 기반의 머신러닝 비교

R

- 통계 전용 프로그램 언어
- 통계 분석 특화 언어 + 다양하고 많은 통계 패키지
- 개발 언어 익숙 **✗**, 통계 분석 능한 현업 사용자 추천

파이썬

- 다양한 영역에서 사용되는 개발 전문 프로그램 언어
- 직관적인 문법 & 객체지향 & 함수형 프로그래밍 모두 포괄하는 유연한 프로그램 아키텍처 + 다양한 라이브러리
- 머신러닝 시작하려는 사람. 특히 개발자 추천
- 딥러닝 프레임워크 - 텐서플로 (TensorFlow), 케라스(Keras), 파이토치(PyTorch) 등 - 파이썬 우선 정책 지원.

02. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

- 머신러닝 패키지
 - 사이킷런(Scikit-Learn) : 가장 대표적인 머신러닝 패키지. 데이터 마이닝 기반
- 행렬/선형대수/통계 패키지
 - 넘파이(NumPy) : 행렬, 선형대수 다루는 패키지
 - 사이파이(SciPy) : 자연과학과 통계 위한 다양한
- 데이터 핸들링
 - 판다스 : 대표적인 데이터 처리 패키지. 2차원 데이터 처리 특화
- 시각화
 - 맷플롯립(Matplotlib) : 대표적인 파이썬 시각화 패키지. 세분화된 API, 디자인 투박, 효율 ↓
 - 시본(Seaborn) : 맷플롯립 보완하여 함축적으로 만들
- 주피터 노트북(Jupyter Notebook): 대화형 파이썬 툴인 아이파이썬(Python, Interactive Python) 지원 툴

03. 넘파이

= Numerical Python. 넘파이 (NumPy)

- 선형대수 기반 프로그램 쉽게 만들 수 있도록 지원.
- 루프 사용 없이 대량 데이터 배열 연산 가능 → 빠른 연산 속도 보장
- C/C++과 같은 저수준 언어 기반의 호환 API 제공 - 데이터 주고받거나 API 호출해 쉽게 통합 가능
- 다양한 데이터 핸들링 기능 제공 but 판다스만큼 편리하진 **×**

1) 넘파이 ndarray 개요

import numpy as np : 넘파이 모듈 임포트

→ as np 추가해 약어로 모듈 표현하는게 관례

- ndarray : 넘파이의 기반 데이터 타입.
- 다차원(Multi dimension) 배열 쉽게 생성 & 다양한 연산 수행

array() 함수

리스트 등 다양한 인자 입력받아서 ndarray로 변환.

```
▶ array1 = np.array( [ 1, 2, 3 ] )  
array2 = np.array( [ [ 1, 2, 3 ], [ 2, 3, 4 ] ] )  
array3 = np.array( [ [ 1, 2, 3 ] ] )
```

shape 변수

생성된 ndarray 크기, 즉 행과 열 개수 튜플(tuple) 형태로 가짐 → ndarray 배열 차원 알 수 있음.

```
▶ print ( array1.shape )  
print( array2.shape )  
print( array3.shape )  
  
>>> ( 3, )    # 1차원 3개 데이터  
( 2, 3 )    # 2차원, 2행 3열 데이터
```

(1, 3) # 2차원, 1행 3열 데이터

⇒ array1과 array3은 데이터값은 동일하나 차원 다름

ndim

array의 차원 확인

array1.ndim

2) ndarray의 데이터 타입

- ndarray 내 데이터값 - 숫자, 문자열, 불 값 등 모두 가능
- ndarray 내 데이터 타입은 연산 특성상 같은 데이터 타입만 가능

dtype 속성

ndarray 내 데이터 타입 확인

```
array1.dtype
```

if 다른 데이터 유형 섞여있는 list를 ndarray로 변환

⇒ 데이터 크기가 더 큰 데이터 타입으로 형 변환 일괄 적용

astype() 메소드

ndarray 내 데이터값 타입 변경

인자로 원하는 타입을 **문자열**로 지정

메모리 절약해야 할 때 이용 - 대용량 데이터의 ndarray 만들 때 많은 메모리 사용되기 때문

```
array_int = np.array([1, 2, 3])  
array_float = array_int.astype('float64')
```

3) ndarray를 편리하게 생성하기 — arange, zeros, ones

- 특정 크기&차원 가진 ndarray → 연속값이나 0 or 1로 초기화해 쉽게 생성할 필요 있는 경우
- 주로 테스트용 데이터 만들거나 대규모 데이터 일괄적 초기화 할 경우

arange() 함수	array를 range()로 표현한 것	np.arange(10)
-------------	-----------------------	---------------

zeros() 함수	인자로 튜플 형태 shape 값 입력 → 모든 값 0으로 채운 해당 shape 가진 ndarray 반환	<code>np.zeros((3, 2), dtype = 'int32')</code>
ones() 함수	인자로 튜플 형태 shape 값 → 모든 값 1로 채운 ~	<code>np.ones((3, 2))</code>

`zeros()`, `ones()` 모두 데이터 형 default = float64

4) ndarray의 차원과 크기를 변경하는 reshape()

reshape() 메소드	ndarray 특정 차원 및 크기로 변환 함수 인자로 원하는 크기 부여 but 지정된 사이즈로 변경 불가능하면 오류 발생 인자 -1 : 원래 ndarray와 호환되는 새로운 shape로 변환 <code>reshape(-1, 1)</code> : 2차원, 여러개 행&1개 열 가진 ndarray로 변환 보장	<code>array1.reshape(2, 5)</code> <code>array1.reshape(-1, 5)</code>
tolist() 메소드	ndarray를 리스트 자료형으로 변환 : 시각적으로 더 이해하기 쉬울 수 있기 때문	<code>array5.tolist()</code>

5) 넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱(Indexing)

- 특정한 데이터만 추출 : 원하는 위치 인덱스 값 지정 → 해당 위치 데이터 반환
- 슬라이싱 : 연속된 인덱스상의 ndarray 추출
- 팬시 인덱싱(Fancy Indexing) : 일정한 인덱싱 집합을 리스트 또는 ndarray 형태로 지정해 해당 위치에 있는 데이터의 ndarray를 반환
- 불린 인덱싱(Boolean Indexing) : 특정 조건에 해당하는지 여부인 True/False 값 인덱싱 집합을 기반으로 True에 해당하는 인덱스 위치에 있는 데이터의 ndarray 반환

(1) 단일 값 추출

해당 위치 인덱스 값 [] 안에 입력 → 1개 데이터값 추출
단일 인덱스 이용하여 ndarray 내 데이터값도 수정 가능

다차원 : 2차원의 경우 [행, 열] 인덱스 통해 접근

넘파이에선 로우, 칼럼 방식 사용 **✗**.

정확한 표현 : **axis 0 = 로우방향 축, axis1 = 칼럼 방향 축**

(2) 슬라이싱

' : ' 기호 이용해 연속한 데이터 슬라이싱해서 추출. **ndarray 타입 반환**



시작, 종료 인덱스 생략 가능

1. ':' 기호 앞에 시작 인덱스를 생략하면 자동으로 맨 처음 인덱스인 0으로 간주합니다.
2. ':' 기호 뒤에 종료 인덱스를 생략하면 자동으로 맨 마지막 인덱스로 간주합니다.
3. ':' 기호 앞/뒤에 시작/종료 인덱스를 생략하면 자동으로 맨 처음/맨 마지막 인덱스로 간주합니다

2차원 슬라이싱

- 콤마(,)로 로우와 칼럼 인덱스를 지칭.
 - 한쪽에만 슬라이싱 & 다른 쪽 단일 값 인덱스도 가능
 - 2차원 ndarray 뒤에 오는 인덱스 없애면 1차원 ndarray 반환
- `array2d[ 0:2, 2:0 ]`
 - `array2d[ :2, 0 ]`
 - `array2d[ 0 ]`

(4) 팬시 인덱싱

리스트나 ndarray로 인덱스 집합 지정 → 해당 위치 인덱스에 해당하는 ndarray 반환

<code>array2d[[0,1], 2]</code>	<code>array2d[[0, 1], 0:2]</code>	<code>array2d[[0, 1]]</code>
1 2 3	1 2 3	1 2 3
4 5 6	4 5 6	4 5 6
7 8 9	7 8 9	7 8 9

`array2d[ [ 0, 1 ] ] ⇒ ( (0, :), (1, : ) )` 인덱싱 적용

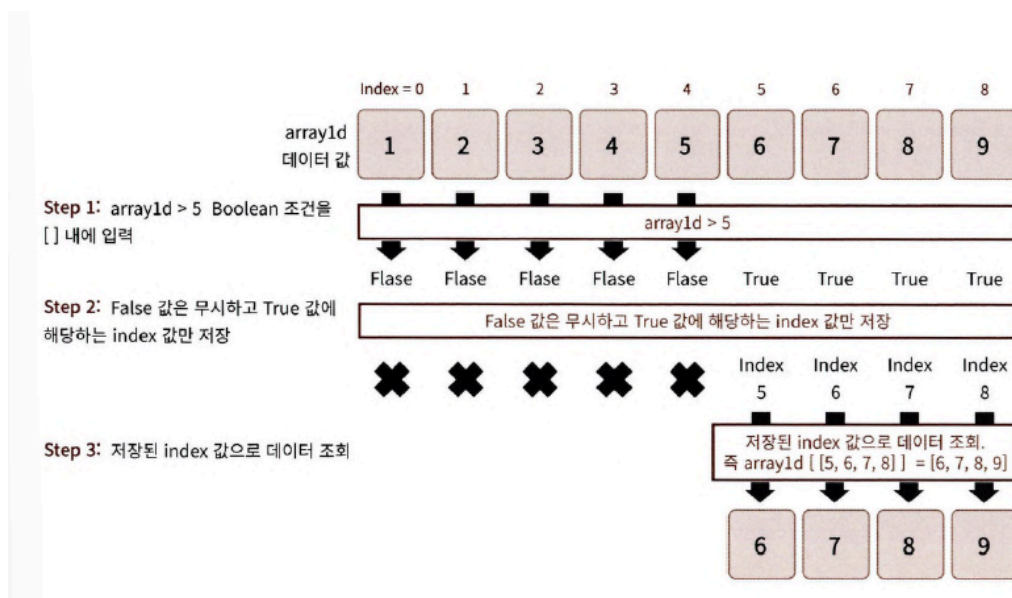
(5) 불린 인덱싱

조건 필터링 + 검색 동시에 가능

ndarray의 인덱스 지정하는 [] 내에 원하는 필터링 조건문 기재 → 해당 조건 만족하는 ndarray 데이터 세트 반환

```
array1d[ array1d > 5]
```

불린 인덱싱 동작하는 단계 도식화

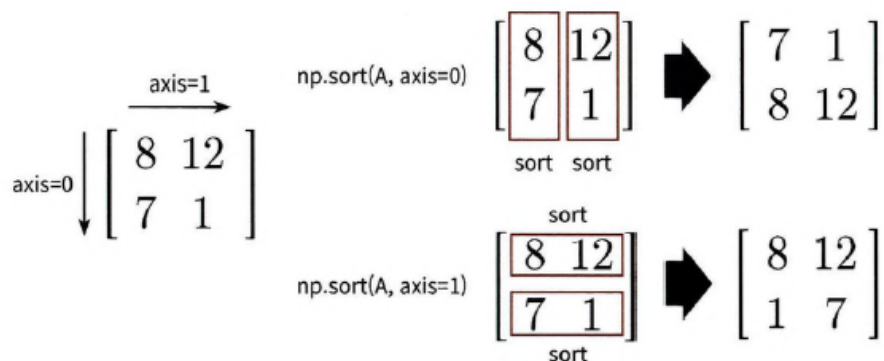


6) 행렬의 정렬 - sort()와 argsort()

(1) 행렬 정렬

<code>np.sort()</code>	넘파이에서 sort() 호출	원 행렬 그대로 유지 → 원 행렬의 정렬된 행렬 반환	<code>np.sort(org_array)</code>
<code>ndarray.sort()</code>	행렬 자체에서 sort() ghcnf	원 행렬 자체를 정렬한 형태로 변환 → None 값 반환	<code>org_array.sort()</code>

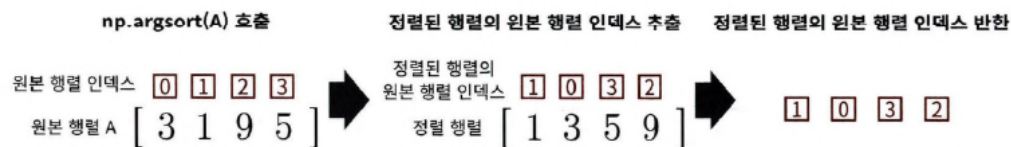
- `np.sort()` 와 `ndarray.sort()` 모두 기본적으로 오름차순으로 행렬 내 원소 정렬 내림차순 정렬 위해서는 [: : -1] 적용
- 2차원 이상 행렬 : axis 축 값 설정 통해 로우 방향 or 칼럼 방향 정렬 수행



(2) 정렬된 행렬의 인덱스를 반환하기

np.argsort()

- 정렬 행렬의 원본 행렬 인덱스를 ndarray 형으로 반환



- 내림차순 정렬 시 원본 행렬 인덱스 - `np.argsort()[::-1]`
- 넘파이의 ndarray는 메타 데이터 가질 수 없음 → if 학생별 시험 성적 데이터로 표현 위 해선 학생 이름과 시험성적 각각 별도의 ndarray로 가져야함 → 한 array의 바뀐 정렬 다른 array에 적용하여 데이터 추출할 때 많이 사용됨

7) 선형대수 연산 - 행렬 내적과 전치 행렬 구하기

(1) 행렬 내적(행렬 곱)

np.dot()

- A행렬과 B행렬 내적 연산 : `np.dot( A, B )`

(2) 전치 행렬

transpose()

- 전치행렬 : 원 행렬의 행과 열 위치 교환한 원소로 구성된 행렬
- `np.transpose(A)`

04. 데이터 핸들링 - 판다스

- 판다스는 행과 열로 이루어진 2차원 데이터를 효율적으로 가공/처리 가능한 다양한 기능 제공
- 넘파이보다 훨씬 유연하고 편리하게 데이터 핸들링 가능
- csv, tab 같은 다양한 유형의 분리 문자로 칼럼을 분리한 파일 DataFrame으로 손쉽게 로딩

DataFrame

- 판다스의 핵심 객체
- 여러개의 행과 열로 이뤄진 2차원 데이터 담는 데이터 구조체
- **칼럼 여러 개**인 데이터 구조체
- 여러개의 Series로 이루어짐

Index

- RDBMS의 PK처럼 개별 데이터를 고유하게 식별하는 Key 값
- 단 하나의 칼럼으로 구성된 데이터 세트

Series

- Series와 DataFrame은 모두 Index를 key 값으로 가짐
- **칼럼이 하나**인 데이터 구조체

1) 판다스 시작 - 파일을 DataFrame으로 로딩, 기본 API

판다스는 다양한 포맷으로 된 파일을 DataFrame으로 로딩할 수 있는 편리한 API 제공

<code>read_csv()</code>	csv 파일 포맷 변환 위한 API 필드 구분 문자(Delimiter)가 콤마(',')	어떤 필드 구분 문자 기반 파일 포맷도 DataFrame으로 변환 가능 → sep에 해당 구분 문자 입력 <code>pd.read_csv('파일명', sep = '\t')</code>
<code>read_table()</code>	필드 구분 문자(Delimiter)가 탭('\t')	
<code>read_fwf()</code>	Fixed Width, 즉 고정 길이 기반 칼럼 포맷을 DataFrame으로 로딩하기 위한 API	

모든 DataFrame 내의 데이터는 생성되는 순간 고유의 Index 값 가짐.

<code>DataFrame.head()</code>	DataFrame의 맨 앞에 있는 N개의 로우를 반환 default는 5개	<code>titanic_df.head(3)</code>
<code>shape</code> 변수	DataFrame의 행과 열을 튜플 형태로 반환	<code>titanic_df.shape</code>
<code>info()</code> 메소드	총 데이터 건수, 데이터타입, Null 건수	<code>titanic_df.info()</code>

<code>describe()</code> 메소드	칼럼별 숫자형 데이터값의 n-percentile 분포 도, 평균값, 최댓값, 최솟값 나타냄 오직 숫자형(int, float 등) 칼럼 의 분포 도만 조사 object 타입 칼럼 출력 자동 제외 ⇒ 개략적인 분포도 확인할 수 있어 유용	<code>titanic_df.describe()</code>
<code>value_counts()</code> 메소드	지정된 칼럼의 데이터값 유형건수 반환 많은 건수 순서로 정렬 반환 데이터 타입 Series 객체	<code>titanic_df['Pclass'].value_counts()</code>
<code>DataFrame['칼럼명']</code>	해당 칼럼 Series 객체를 반환	<code>titanic_df['Pclass']</code>

2) DataFrame과 리스트 딕셔너리, 넘파이 ndarray 상호 변환

DataFrame은 파이썬의 리스트, 딕셔너리, 넘파이 ndarray 등으로 변환 가능
사이킷런의 많은 API는 넘파이 ndarray 입력인자로 사용 → 상호 변환 빈번

(1) 넘파이 ndarray, 리스트, 딕셔너리를 DataFrame으로 변환하기

DataFrame

칼럼명 가짐 → 상대적으로 편하게 데이터 핸들링 가능
일반적으로 변환 시 칼럼명 지정 (지정 안하면 자동 할당)

- 생성 인자 data : 리스트, 딕셔너리, 넘파이 ndarray 입력받음
- 생성 인자 columns : 칼럼명 리스트 입력받음
 - 리스트, 넘파이 ndarray를 변환
 - 1차원 데이터는 칼럼 1개만 필요
 - 2차원 데이터는 열 개수만큼 칼럼 필요
 - 딕셔너리를 변환
 - 키(Key)는 칼럼명 - 문자열 형태
 - 값(Value)은 키에 해당하는 칼럼 데이터로 변환 - 리스트(또는 ndarray) 형태

(2) DataFrame을 넘파이 ndarray, 리스트, 딕셔너리로 변환하기

<code>values</code>	DataFrame을 넘파이 ndarray로 변환	<code>df_dict.values</code>
---------------------	----------------------------	-----------------------------

<code>tolist()</code>	리스트로의 변환 : values로 얻은 ndarray에 tolist() 호출	<code>df_dict.values.tolist()</code>
<code>to_dict()</code> 메소드	딕셔너리로의 변환 인자로 'list' 입력하면 딕셔 너리 값 리스트형으로 반환	<code>df_dict.to_dict('list')</code>

3) DataFrame의 칼럼 데이터 세트 생성과 수정

데이터 세트 생성&수정 → [] 연산자 이용

- DataFrame [] 내에 새로운 칼럼명 입력 + 값 할당 ⇒ Series가 기존 DataFrame에 추가됨
`titanic_df['Age_0'] = 0`
Series에 상숫값 할당 → Series의 모든 데이터 세트에 일괄 적용
- 기존 칼럼 Series 가공 → 새로운 칼럼 Series 만듦
DataFrame [] 내 업데이트 원하는 칼럼명 입력 후 값 할당 ⇒ 기존 칼럼 값 일괄 업데이트
`titanic_df['Age_by_10'] = titanic_df['Age_by_10'] + 100`

4) DataFrame 데이터 삭제

- `drop()` 메소드
 - axis 파라미터
 - axis = 0 입력 : 로우 축 방향 드롭 수행. 특정 로우 드롭
labels에 오는 값 = 인덱스 = 특정 로우
주로 이상치 데이터 삭제 경우
 - axis = 1 입력 : 칼럼 축 방향 드롭 수행. 지정 칼럼 드롭
labels에 원하는 칼럼명 + axis = 1 입력 → 지정된 칼럼 드롭
대부분 기존 칼럼 값 가공 후 삭제하는 경우
 - labels 파라미터
 - inplace 파라미터
 - Inplace=False : 자기자신의 DataFrame 데이터 삭제 ✗, 삭제된 결과 DataFrame 반환. **default 값**
 - inplace=True : 자신의 DataFrame 데이터 삭제. 반환 값 None
if 여러 개 칼럼 삭제 : labels 파라미터로 삭제하려는 칼럼명 리스트 형태로 입력

5) Index 객체

- Index 객체
 - DataFrame, Series의 레코드를 고유하게 식별
 - RDBMS의 **PK(Primary Key)**와 유사
 - 식별용으로만 사용, 연산에서는 제외
- Index 확인
 - `DataFrame.index` / `Series.index`
 - `Index.values` → 실제 값을 1차원 ndarray로 확인
 - 단일 값 반환, 슬라이싱 가능
- Index 변경
 - 한 번 만들어진 Index 객체는 변경할 수 없음
- `reset_index()`
 - 연속된 int 숫자형 인덱스로 새롭게 할당
 - 기존 인덱스 → 'index' 칼럼으로 추가
 - 연속된 int 숫자형 인덱스로 다시 만들 때 사용
- Series에 `reset_index()`
 - 기존 인덱스가 칼럼으로 추가 → DataFrame 반환
 - `drop=True`
 - 기존 인덱스 삭제
 - 새로운 칼럼이 추가되지 않음 → Series 유지

6) 데이터 셀렉션 및 필터링

(1) DataFrame의 [] 연산자

- 기본적으로 칼럼 지정 연산자
- `[ ]` 안에 입력 가능
 - 칼럼명
 - 칼럼명의 리스트
 - 불린 인덱싱

- `DataFrame[0]`, `DataFrame[[0, 1, 2]]` → 컬럼명이 아니므로 오류
- 슬라이싱을 이용한 데이터 추출도 가능하지만 **사용하지 않는 것이 좋음**

(2) DataFrame `iloc[ ]` 연산자

- 위치 기반 인덱싱
- 행과 열의 **위치 정숫값**으로 데이터 선택
- 정수형 슬라이싱, 팬시 인덱싱 가능
- **불린 인덱싱 불가능**
- `1` → 마지막 데이터
- 인덱스 값이나 컬럼명을 입력하면 오류

(3) DataFrame `loc[ ]` 연산자

- **명칭(Label) 기반 인덱싱**
- 행 위치 → **DataFrame의 인덱스 값**
- 열 위치 → **컬럼명**
- 불린 인덱싱 가능
- 슬라이싱 → **종료 값까지 포함**

주의할 점

1. 개별 또는 여러 컬럼 값 전체를 추출하고자 한다면 `iloc[ ]`나 `loc[ ]`를 사용하지 않고 `data_df['Name']`과 같이 `DataFrame['컬럼명']`만으로 충분합니다. 하지만 행과 열을 함께 사용하여 데이터를 추출해야 한다면 `iloc[ ]`나 `loc[ ]`를 사용해야 합니다.
2. `iloc[ ]`와 `loc[ ]`를 이해하기 위해서는 명칭 기반 인덱싱과 위치 기반 인덱싱의 차이를 먼저 이해해야 합니다. DataFrame의 인덱스나 컬럼명으로 데이터에 접근하는 것은 명칭 기반 인덱싱입니다. 0부터 시작하는 행, 열의 위치 좌표에만 의존하는 것이 위치 기반 인덱싱입니다.
3. `iloc[ ]`는 위치 기반 인덱싱만 가능합니다. 따라서 행과 열 위치 값으로 정수형 값을 지정해 원하는 데이터를 반환합니다.
4. `loc[ ]`는 명칭 기반 인덱싱만 가능합니다. 따라서 행 위치에 DataFrame 인덱스가 오며, 열 위치에는 컬럼명을 지정해 원하는 데이터를 반환합니다.
5. 명칭 기반 인덱싱에서 슬라이싱을 '시작점:종료점'으로 지정할 때 시작점에서 종료점을 포함한 위치에 있는 데이터를 반환합니다.

(4) 불린 인덱싱

- 조건을 이용하여 원하는 데이터를 필터링
- `[ ]`, `loc[ ]` 에서 지원
- `iloc[ ]` 에서는 지원하지 않음
- 복합 조건 연산자
 - **AND** → `&`
 - **OR** → `|`
 - **NOT** → `~`
- 개별 조건은 `()` 로 묶음
- 조건을 변수에 할당하여 결합 가능

7) 정렬, Aggregation 함수, GroupBy 적용

(1) DataFrame, Series의 정렬 - `sort_values()`

- DataFrame과 Series의 정렬 → `sort_values()`
- RDBMS SQL의 **ORDER BY**와 유사
- 주요 파라미터
 - `by` → 정렬할 칼럼
 - `ascending=True` → 오름차순
 - `ascending=False` → 내림차순
 - `inplace=True` → 정렬 결과를 기존 DataFrame에 적용
 - `inplace=False` → 기존 DataFrame 유지 + 정렬된 결과 반환
- 기본값
 - `ascending=True`
 - `inplace=False`
- 여러 칼럼으로 정렬 → `by` 에 칼럼 리스트 입력

(2) Aggregation 함수 적용

- 주요 Aggregation 함수
 - `min()`

- `max()`
- `sum()`
- `count()`
- DataFrame에 직접 Aggregation 함수 적용
 - 모든 칼럼에 적용
- `count()`
 - Null 값을 반영하지 않음
 - Null 값이 있는 칼럼은 결과값이 다를 수 있음
- 특정 칼럼에 Aggregation 함수 적용
 - 대상 칼럼만 추출한 후 Aggregation 함수 적용

(3) `groupby()` 적용

- RDBMS SQL의 **GROUP BY**와 유사
- `by` 에 칼럼을 입력 → 해당 칼럼 기준으로 GroupBy
- `groupby()` 결과 → **DataFrameGroupBy 객체**
- GroupBy 후 Aggregation 함수 적용
 - `groupby()` 대상 칼럼을 제외한 모든 칼럼에 적용
 - 특정 칼럼에만 적용 → 원하는 칼럼을 필터링한 후 Aggregation 함수 적용
- 여러 Aggregation 함수 적용
 - `agg()` 사용
 - `agg()` 안에 여러 Aggregation 함수명을 입력
- 여러 칼럼에 서로 다른 Aggregation 함수 적용
 - `agg()` 에 딕셔너리 형태로 입력
 - 각 칼럼과 적용할 Aggregation 함수를 지정

8) 결손 데이터 처리하기

(1) `isna()` 로 결손 데이터여부 확인

- 결손 데이터(Missing Data)

- 값이 없는 경우 → **NULL**
- NumPy의 **NaN**으로 표시
- 머신러닝 알고리즘은 기본적으로 **NaN 값을 처리하지 못함**
- NaN 값은 **평균, 총합 등의 함수 연산에서 제외**
- `isna()` → 데이터가 **NaN인지 여부 확인**
 - `True` → NaN
 - `False` → NaN 아님
- 결손 데이터 개수 → `isna()` 결과에 `sum()` 적용
 - `True` → 1
 - `False` → 0

(2) `fillna()`로 결손 데이터 대체하기

- `fillna()` → **NaN 값을 다른 값으로 대체**
- 실제 데이터 변경 방법
 - `fillna()`의 반환 값을 다시 받기
 - `inplace=True` 설정
- 타이타닉 데이터 결손값 처리
 - **Cabin** → `'C000'`으로 대체
 - **Age** → 평균 나이로 대체
 - **Embarked** → `'S'`로 대체

9) `apply` `lambda` 식으로 데이터 가공

`apply` `lambda`

- `apply` + `lambda` → **DataFrame이나 Series의 레코드별 데이터 가공**
- 복잡한 데이터 가공이 필요할 경우 사용
- 칼럼에 일괄적으로 데이터를 가공하는 것보다 **속도 면에서 느림**

`lambda` 식

- 함수의 선언과 처리 과정을 한 줄의 식으로 표현
- `:` 기준
 - 왼쪽 → 입력 인자
 - 오른쪽 → 계산식 및 반환값
- 여러 개의 값을 입력 인자로 사용할 경우 → `map()`과 결합

lambda 식의 if else

- `if`, `else` 지원
- **if 절**
 - 반환값을 조건식보다 먼저 작성
- **else 절**
 - `else` 뒤에 반환값 작성
- `else if` 는 지원하지 않음
 - `else` 안에 `if else` 를 내포하여 사용
- `else if` 가 많거나 switch case문과 같은 경우 → 별도의 함수 생성이 더 적합